Questions for Dr. Lim

1. How many nodes do we have here?

2. For edges, are node IDs stored in this list? What do -1s represent?

3. How do we tell what phase a node is in?

4. What about the size attribute?

5. Go through directValueOpt.

- We have 93 nodes.
  - Constructs a real node and creates a unique ID and is time-based.
- No information telling us what phase we are in.
- Each phase can make both manipulations.
- If "Alive", node is converted into a machine ID.
- ~~Phases have IDs.~~
  - Mark the beginning of phases.
- Look at nodes created before an instruction ID.
- Function that creates a new node: 96937
- Tracer might make a mistake and not specify
- Python script to map this out
- fnCallRet... gives us a timeline → keep track of

ASLR

returns, not calls.
  - Called by the function before it.
- Will have to extract the phase information.
- Key-value maps, need to see when things are executed.
- which instruction IDs happen before function execution
- Instruction ID and function ID are different.
  - Show the order of execution.
- Phases end in Phase::Run
- Look at Instruction ID to determine order.
- 96937 → Compiler::Node:New
- instAccess → All instructions that access a specific node.
- type → different type accesses.

- 96937 → Bugs may happen when nodes are incorrectly created.
  - Might be created too late or created with wrong data.

- grep "Phase::Run" json file
- where is key being used?
- Not fixed.
- where are addressees also being accessed?
- Nodes can be incorrectly removed from edges.
- -1 in an edge means that the edge has been removed.
- Only look at one node at a time.

- Overview graph of each phase?
  - or show one node one at a time?
- Don't want to show everything at once!

- 15-20 phases
- For -1, will have to go to remove_edge to figure out which node was disconnected and display that.